simple_viz — project handoff

A complete, self-contained guide to this assignment so you can continue any time, even without the original chat. Read top to bottom once; after that use it as a reference.

---

1. What this project is

`simple_viz` is a small Python **data-visualization library** built for a communication course. It makes "communication-first" charts that tell one data story — **Pinterest's 2025 results** — where every chart is the right chart for one specific question and makes deliberate design choices.

The assignment has two graded parts:
1. **The library** — a real, installable, importable Python package.
2. **A ~2-minute video** demoing it (see §8 and `VIDEO_SCRIPT.md`).

The core idea / thesis of the data story: Pinterest has a huge, fast-growing *global* audience, but its *revenue* is heavily concentrated in the US & Canada — "users are global, revenue is domestic."

---

2. Folder structure

```
Comms_Data_Science/
├─ src/simple_viz/              # THE LIBRARY (this is what's graded)
│  ├─ __init__.py          # public API: exposes the 5 chart functions + __version__
│  ├─ core.py              # the 5 chart functions (the real logic)
│  └─ theme.py             # shared colours + styling helpers (one design system)
├─ tests/
│  └─ test_simple_viz.py        # 9 pytest tests (return types + 2 calculations)
├─ data/                   # real, sourced Pinterest data (CSV)
│  ├─ pinterest_mau.csv               # annual Q4 MAU 2019–2025
│  ├─ pinterest_mau_quarterly.csv     # quarterly MAU (28 points) 2019–2025
│  ├─ pinterest_regions_q4_2025.csv   # MAU + revenue by region (Q4 2025)
│  ├─ pinterest_revenue.csv           # annual revenue 2019–2025
│  └─ SOURCES.md                      # where every number comes from (links)
├─ examples/               # DEMOS (not part of the library)
│  ├─ simple_viz_gallery.py              # renders the 5 charts + a combined PDF
│  ├─ simple_viz_poster.py               # the one-page infographic poster
│  ├─ simple_viz_animation.py            # animated growth line  → simple_viz_growth.gif
│  ├─ simple_viz_story_animation.py      # full story animated   → simple_viz_story.gif
│  ├─ simple_viz_01_big_number.png … 05_revenue_per_user.png   # the 5 chart images
│  ├─ simple_viz_gallery.pdf             # 5 charts combined
```

```
    │   ├─ simple_viz_poster.png / .pdf        # the poster
    │   ├─ simple_viz_growth.gif               # animated growth line
    │   └─ simple_viz_story.gif                # animated full story
    ├─ README.md                # install + usage + the function/chart-type table
    ├─ ANALYSIS.md              # the business "so what": purpose, insights, decisions
    ├─ VIDEO_SCRIPT.md          # timed 2-minute recording script
    ├─ HANDOFF.md               # this file
    ├─ pyproject.toml           # package metadata + dependencies
    ├─ LICENSE                  # MIT
    └─ .github/workflows/python-publish.yml   # (auto-added) publish-to-PyPI on release
```

Install name vs import name: it's **published on PyPI** as `simple-viz-prashasti` (that's the `pip install` name), and it **imports** as `simple_viz`. Live page: https://pypi.org/project/simple-viz-prashasti/ . Version is `0.1.0` (first working version). Install with `pip install simple-viz-prashasti`, or from source with `pip install -e .`.

---

3. The library API (the 5 chart functions)

All live in `src/simple_viz/core.py`. Each takes a pandas DataFrame (or simple values) and **returns a matplotlib `Figure`** you can `savefig` in any format.

Function	Chart type	What it answers	Key call
`big_number(value, unit, label, footnote="")`	Single important number	How big is the headline?	`simple_viz.big_number(619, "M", "monthly active users")`
`growth_line(df, x, y, title, annotate=None)`	Change over time	Which way is the trend going?	`simple_viz.growth_line(mau, "year", "maus_millions", title=..., annotate=(2021,"dip"))`
`revenue_bar(df, category, value, title, spotlight=None)`	Comparison	Who contributes the most?	`simple_viz.revenue_bar(reg, "region", "revenue_musd", title=..., spotlight="US & Canada")`
`share_gap(df, category, part_a, part_b, title, highlight=None)`	Parts of a whole	Users vs revenue mismatch	`simple_viz.share_gap(reg, "region", "maus_millions", "revenue_musd", title=..., highlight="Rest of World")`
`revenue_per_user(df, category, revenue, users, title, spotlight=None)`	Comparison / ratio	How well is each region monetised?	`simple_viz.revenue_per_user(reg, "region", "revenue_musd", "maus_millions", title=...)`

---

4. How the code works (module by module)

`src/simple_viz/theme.py` — one shared design system

Holds the palette (`PIN_RED = #E60023`, plus muted greys `MUTED`, `INK`,

`SUBTLE`, `GRID`) and three helpers every chart reuses:
- `apply_base_style(ax)` — removes chartjunk (drops top/right spines, softens the rest, kills tick marks, lightens the y-gridlines).
- `titles(ax, title, subtitle)` — left-aligned bold **takeaway** title + a lighter subtitle (leads with the message, not the variable name).
- `new_axes(figsize)` — makes a figure/axes with headroom for the titles.
- It also sets `plt.rcParams["text.parse_math"] = False` — see §7, problem 2.

`src/simple_viz/core.py` — the 5 functions
Each function: builds a figure, draws its chart, applies the shared theme, and returns the `Figure`. `revenue_bar` and `revenue_per_user` share one private helper `_spotlight_barh` (sorted bars, one highlight colour, end-of-bar labels) so there's no duplicated code. `revenue_per_user` computes revenue ÷ users itself so the ratio is never hand-entered.

`src/simple_viz/__init__.py` — the public face
Imports the 5 functions so users can call `simple_viz.growth_line(...)`, and defines `__all__` and `__version__`.

The design rules every chart follows
1. **One highlight colour** (Pinterest red); everything else muted grey — colour directs attention, doesn't decorate.
2. **Takeaway titles** — "Rest of World is 58% of users but 7% of revenue," not "Users and revenue by region."
3. **Honest axes** — bars/lines start at zero; shares normalised to 100%.
4. **No chartjunk** — no top/right spines, no tick marks, light gridlines, values labelled directly (no legends).

---

5. The visualizations produced

Five library charts (run `python examples/simple_viz_gallery.py`):
1. `simple_viz_01_big_number.png` — **619M** big-number callout.
2. `simple_viz_02_growth_line.png` — MAU growth 2019–2025, 2021 dip annotated.
3. `simple_viz_03_revenue_bar.png` — Q4 revenue by region (US & Canada spotlighted).
4. `simple_viz_04_share_gap.png` — users-vs-revenue split (Rest of World highlighted).
5. `simple_viz_05_revenue_per_user.png` — implied revenue per user by region.

One composed poster (`python examples/simple_viz_poster.py`):
- `simple_viz_poster.png` / `.pdf` — a one-page infographic: red kicker → serif headline → 619M / $4.2B KPIs → quarterly growth line → revenue-growth bars → the user/revenue split → revenue by region → revenue per user → a red insight band ("a US & Canada user is worth about 35× a Rest-of-World user"). Beige background, white cards, one accent colour.

Two animations (optional creative extras):

- `simple_viz_growth.gif` — the growth line drawing in (`simple_viz_animation.py`).
- `simple_viz_story.gif` — the full story building in sequence (`simple_viz_story_animation.py`).

---

6. Data & sources

All numbers are **real**, from Pinterest's Q4 & Full-Year 2025 report (released Feb 12, 2026) and its 10-K. They were transcribed into the CSVs in `data/` and loaded with `pandas.read_csv` — there is **no live API call**. Full source links are in **`data/SOURCES.md`**. Key figures:
- 619M global MAU (+12% YoY); $4.22B FY revenue (+16%); $1.319B Q4 revenue.
- By region (Q4 2025): US & Canada 105M MAU / $979M; Europe 158M / $245M; Rest of World 356M / $96M.
- Annual revenue 2019→2025: $1.14B → $4.22B; global ARPU $3.81 → $7.21.

See **`ANALYSIS.md`** for what the charts *mean* — the business insights and decisions they support (invest in monetising Rest of World / Europe rather than chasing more saturated US users; manage to ARPU-by-region, not just MAU).

---

7. Two problems solved (for the video's "problems" section)

1. **`twine upload` rejected with `403 Forbidden`.** Cause: at the "Enter your API token" prompt I typed `__token__` — but newer twine wants the token *value* there, not the literal word `__token__` (that's only the username in the old flow). **Fix:** enabled two-factor auth on PyPI, created a real API token, and pasted the actual `pypi-...` value → upload succeeded (live at https://pypi.org/project/simple-viz-prashasti/0.1.0/).
2. **`ModuleNotFoundError` / "does not appear to be a Python project".** Cause: the modern `src/` layout means Python can't find the package until it's installed, and I was in the wrong folder / wrong environment. **Fix:** `cd` into the project root, activate the venv, and `python -m pip install -e .` (editable install links the package so imports resolve and edits apply live).

Alternate library bug (swap in if you'd rather show code): dollar-sign labels like `$4.2B` rendered as garbled LaTeX because matplotlib treats a pair of `$` as math mode. **Fix:** `plt.rcParams["text.parse_math"] = False` set once in `theme.py`.

---

8. How to run everything (commands)

```
# from the project root (folder with pyproject.toml)
```

```
python -m pip install -e .              # install the library (editable)
python -c "import simple_viz; print(simple_viz.__all__)"   # confirm it imports

python -m pip install -e ".[dev]"       # install pytest too
python -m pytest -q                     # run the 9 tests (should pass)

python examples/simple_viz_gallery.py       # render the 5 charts + gallery PDF
python examples/simple_viz_poster.py        # render the poster (PNG + PDF)
python examples/simple_viz_animation.py     # render simple_viz_growth.gif
python examples/simple_viz_story_animation.py   # render simple_viz_story.gif

# build a distributable package (optional)
python -m pip install build twine
python -m build                         # makes dist/*.whl and *.tar.gz
python -m twine check dist/*            # should say PASSED
```

Dependencies: the library needs only **matplotlib** and **pandas**. Built with **setuptools**; tested with **pytest**. (The animations use matplotlib's `PillowWriter`, i.e. **Pillow**, which ships with matplotlib.)

---

9. Git / where the code lives

- GitHub repo: **Prashasti9/Comms_Data_Science**
- Everything is on the **`main`** branch (the working branch was merged and deleted). To keep working: edit files, then `git add -A && git commit -m "..." && git push origin main`.
- Note: matplotlib PDFs re-render with different bytes each run, so the two `.pdf` files may show as "modified" after running the scripts even when the content is identical — that's cosmetic; you can `git checkout -- *.pdf` to discard that churn.

---

10. What's left to do (submission checklist)

- [ ] Record the ~2-minute video following **`VIDEO_SCRIPT.md`**: install → import → README → **two favourite charts + aesthetic choices** (`share_gap` and `growth_line`) → **two problems + fixes** (§7).
- [ ] Record in **Zoom → "Record to the Cloud."**
- [ ] In the recording's share settings, **turn OFF the passcode** (anyone with the link can view).
- [ ] **Test the link in an incognito window** — it must open with no sign-in or password prompt. (This is where marks are most often lost.)
- [ ] Paste the link into the **Website URL** field. **Due: Fri, Jul 31, 11:59 PM PDT.**

One-line summary for the video: *"simple_viz is a small Python library built on matplotlib and pandas; each of its five functions is the right chart for one question, telling the story that Pinterest's users are global but its revenue is domestic."*